MILESTONE - 1

RECEIPT & INVOICE DIGITIZER

Team Members

Frontend: Vlavan Pyaserushti

Tech Startup Presentation

Database: Leelakrishna kondapaturi

Ai: Mohammed suhail ahmed khan

Backend and Ai: Shaik Ayisha

Date : 14-02-2026

PROBLEM STATEMENT

Organizations handle a large volume of receipts and invoices on a daily basis. Most small and mid-scale operations still rely on manual data entry and physical record keeping. This leads to:

- Time-consuming processing of invoices
- High risk of human errors in data entry

There is a need for a secure, scalable digital solution that can centralize invoice management, ensure data security, and provide a foundation for automated data extraction in the future.

PROPOSED SOULTION

We developed a secure web-based Receipt & Invoice Digitizer system to centralize invoice management and eliminate manual record handling.

The solution includes:

- React frontend for user interaction
- FastAPI backend for API and business logic
- JWT-based authentication for secure access
- Password hashing for credential protection
- SQLite database for structured data storage

System Architecture

Frontend

React

JavaScript

Axios for API calls

BACKEND

FastAPI

JWT Authentication

Password hashing

REST APIs

Database:

Sqlite

frontend

React.js

This application is built using React, a JavaScript library used to create single-page applications (SPA).

React allows the UI to update dynamically without reloading the page.

The component LoginPage is a functional component.

JavaScript

JavaScript is used for:

Handling form input

Validating user data (email, name)

Switching between login and signup pages

frontend

React Hooks (useState)

useState is used to manage:

Page state (login, signup, signin)

Form data (name, email, password, language)

It enables state management inside functional components

Tailwind CSS

- Styling is done using Tailwind CSS, a utility-first CSS framework.
- Helps in creating a modern, responsive UI with less CSS code.

frontend

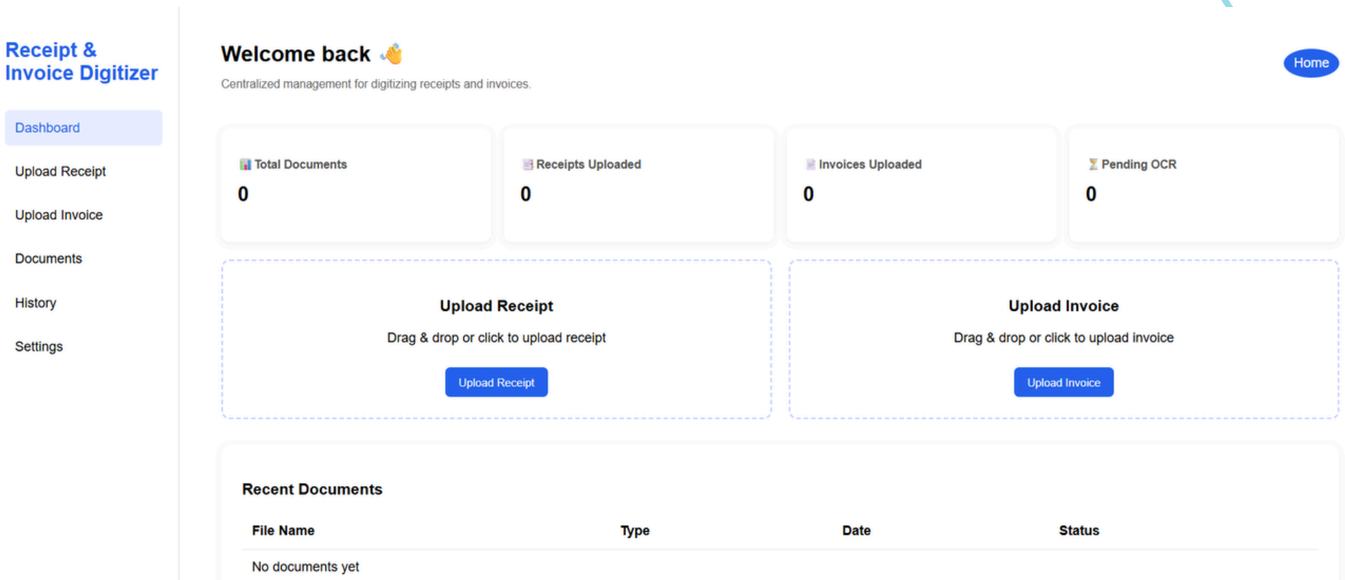
SX (JavaScript XML)

- JSX is used to write HTML inside JavaScript.
- Makes the UI code more readable and structured.

Dashboard

Technologies Used (Frontend)

- React (Functional Components)
- useState for dynamic UI updates
- JSX for structured UI
- CSS for layout and styling
- Event handling for file uploads



- Designed and developed the main dashboard interface using React
- Implemented sidebar navigation and structured layout

Dashboard

- Built dynamic summary cards (Total Docs, Receipts, Invoices, Pending OCR)
- Developed separate Upload Receipt & Upload Invoice sections
- Integrated browser-based file upload functionality
- Created Recent Documents table for displaying uploaded files
- Designed responsive and user-friendly UI with clean CSS styling
- Prepared frontend structure for seamless backend integration

t

Flow of project

KEY FEATURES IMPLEMENTED

- Secure user registration & login
- Password hashing for secure storage
- JWT-based authentication
- Protected API routes
- Basic dashboard UI
- REST API integration

Backend – Database & Security Module

- Designed User Database Schema
- Implemented SQLite with SQLAlchemy
- Implemented Secure Password Hashing
- Integrated Authentication with Database
- Ensured Secure Storage of Credentials

DATABASE STRUCTURE

Field	Type	Description
id	Integer	Primary Key
full_name	String	User Full Name
email	String	Unique Email
hashed_password	String	Hashed Password

USER REGISTRATION FLOW

1)User enters details

2)Password is hashed

3)Data stored in database

4)Database commit executed

small code snippet

```
user = User(
full_name=full_name,
email=email,
hashed_password=hash_password(password)
)
db.add(user)
db.commit()
```

PASSWORD HASHING IMPLEMENTATION

*)Used pbkdf2_sha256 algorithm

*)Implemented using Passlib

*)Prevents storing plain-text passwords

*)Protects against database leaks

Example:$pbkdf2-sha256$29000$...

JWT

- Developed authentication module using FastAPI framework
- Used SQLAlchemy ORM for database interaction
- Generated JWT tokens using python-jose (jose.jwt) library
- Applied HS256 symmetric signing algorithm for token security
- Embedded standard JWT claims:

– sub (subject – user email)

– id (user identifier)

– exp (expiration time using UTC timestamp)

FUTURE SCOPE

- OCR integration
- Automatic data extraction
- Invoice field classification
- CSV / Excel export

THANK YOU